

CS & IT ENGINEERING



Operating System

File System

Lecture - 2

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Inverted & Hashed Page Table

Topic

File System

Topic

Disk Formatting

Topics to be Covered



Topic

Disk Blocks

Topic

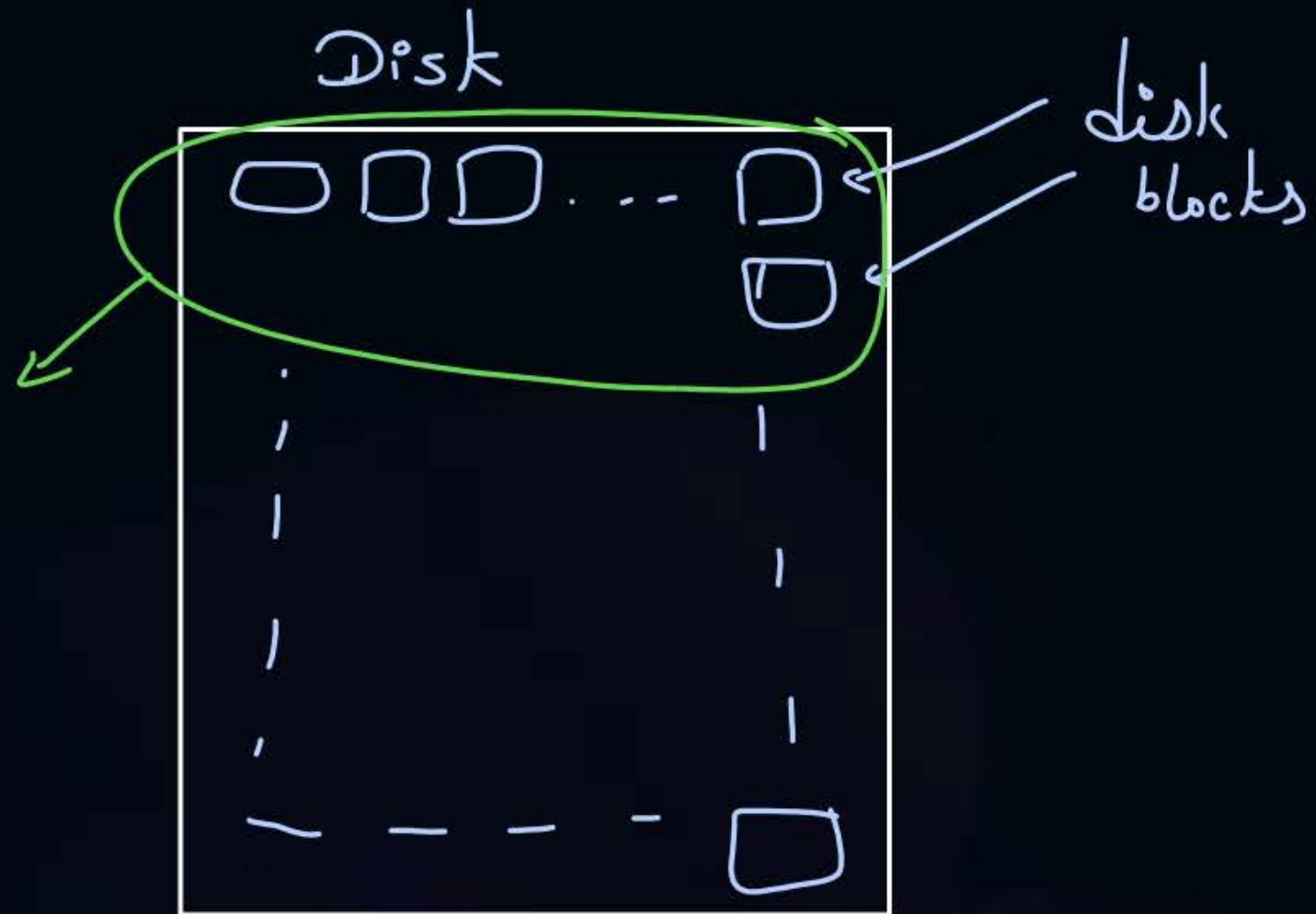
File Allocation Method

Topic

Disk Scheduling



Topic : Disk Blocks



which file is stored on
what blocks



file system maintains
Directories



Topic : Disk Blocks

Number of disk blocks = 2^{16} $\Rightarrow$ disk block no. or disk block address
= 16 bits

Size of each block = 1KB

Total Size of disk? = no. of blocks * block size
= $2^{16} * 1\text{KB}$
= 64 MBytes



Topic : Disk Blocks



Disk block address = 24-bits $\Rightarrow$ no. of blocks = 2^{24}

Size of each block = 2KB

$$\begin{aligned}\text{Total Size of disk?} &= 2^{24} * 2KB \\ &= 2^{35} B \\ &= 32GB\end{aligned}$$



Topic : Disk Blocks



Total disk size = 256GB

Block Size = 2KB

Disk block address?

$$\text{no. of blocks} = \frac{256\text{GB}}{2\text{KB}}$$

$$= \frac{2^{38}}{2^{11}}$$

$$= 2^{27} \Rightarrow \text{disk block add.} = \underline{\underline{27 \text{ bits}}}$$

Ans.



Topic : Free Space Management

→ Now file system maintains infoⁿ about all free blocks.

1. Free List
2. Bitmap Method

1. Free list :- A list of numbers, of the free blocks



2. Bitmap method :- 1-bit infoⁿ about each block is maintained -

for ex:- $\begin{cases} 0 \rightarrow \text{occupied} \\ 1 \rightarrow \text{free} \end{cases}$

001|0000|00000|0001|

↑ ↑
block block
3 4



Topic : Free Space Management

1. No searching in free list, but in bitmap we search for first ~~zero~~ *free block*
2. Free list is faster in allocating a free block
3. Free list size is variable, where as bitmap size is constant



Topic : Question

A particular disk unit uses a bit string to record the occupancy or vacancy of its disk blocks with '0' denoting vacant block and '1' denoting occupied block. A 32-bit part of this string has Hexadecimal value of A3D4B055. The percentage of occupied blocks on the disk for this part is ?

		no. of 1's
A	$\Rightarrow 1010 \Rightarrow$	2
3	$\Rightarrow 0011 \Rightarrow$	2
D	$\Rightarrow 1101 \Rightarrow$	3
4	$\Rightarrow 0100 \Rightarrow$	1
B	$\Rightarrow 1011 \Rightarrow$	3
0	$\Rightarrow 0000 \Rightarrow$	0
5	$\Rightarrow 0101 \Rightarrow$	2
5	$\Rightarrow 0101 \Rightarrow$	2

$$= \frac{15}{32} * 100\%$$

$$= 46.875\%$$

Ques) Consider a disk with 2^{24} blocks, each with 2kB size. The file system maintains a bit map for free space management. No. of blocks required to store bitmap infoⁿ — ?

Solⁿ

for bitmap infoⁿ, file system maintains 1 bit per block.
Hence entire bitmap will have size = 2^{24} bits

$$= 2^{21} \text{ bytes} \\ = 2 \text{ MB}$$

$$\boxed{\text{Ans} = 1024}$$



;



$$\text{no. of blocks needed for bitmap} = \frac{2\text{MB}}{2\text{KB}}$$

$$= 1\text{K} = \underline{\underline{1024}} \text{ Ans}$$

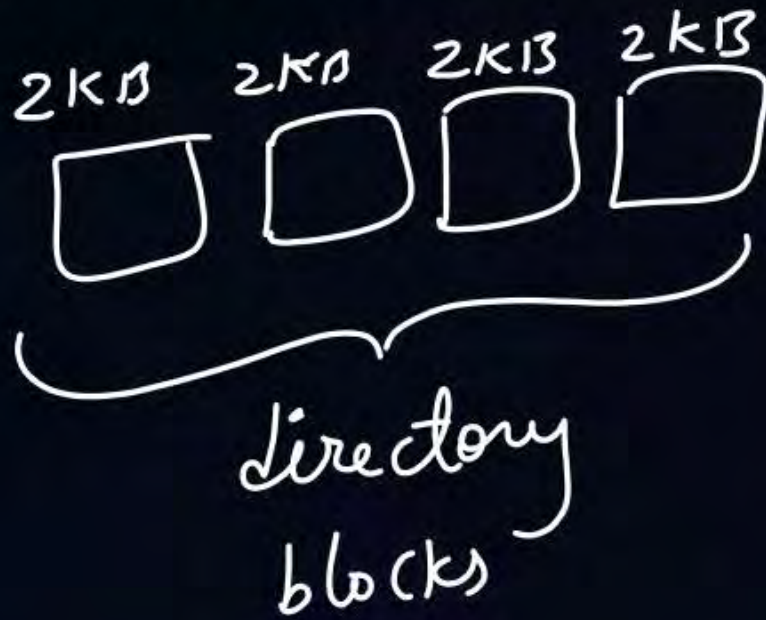


Topic : Question

Contains entries about file

A system directory is kept in 4 disk blocks each of size 2Kbytes. It is a single level-directory and each directory entry is of size 32-bits. $\text{No. of blocks in disk} = 2^{20}$

1. The maximum number of files possible in this system is ? - max no. of directory entries stored on 4 disk blocks
2. The maximum size of any file is?



2) No. of blocks remaining for files = $(2^{20} - 4)$

max file size = $(2^{20} - 4) * 2KB$

$$\frac{8KB}{4B} = 2k$$



Topic : File Allocation Method

↓
how a file must be stored on disk block



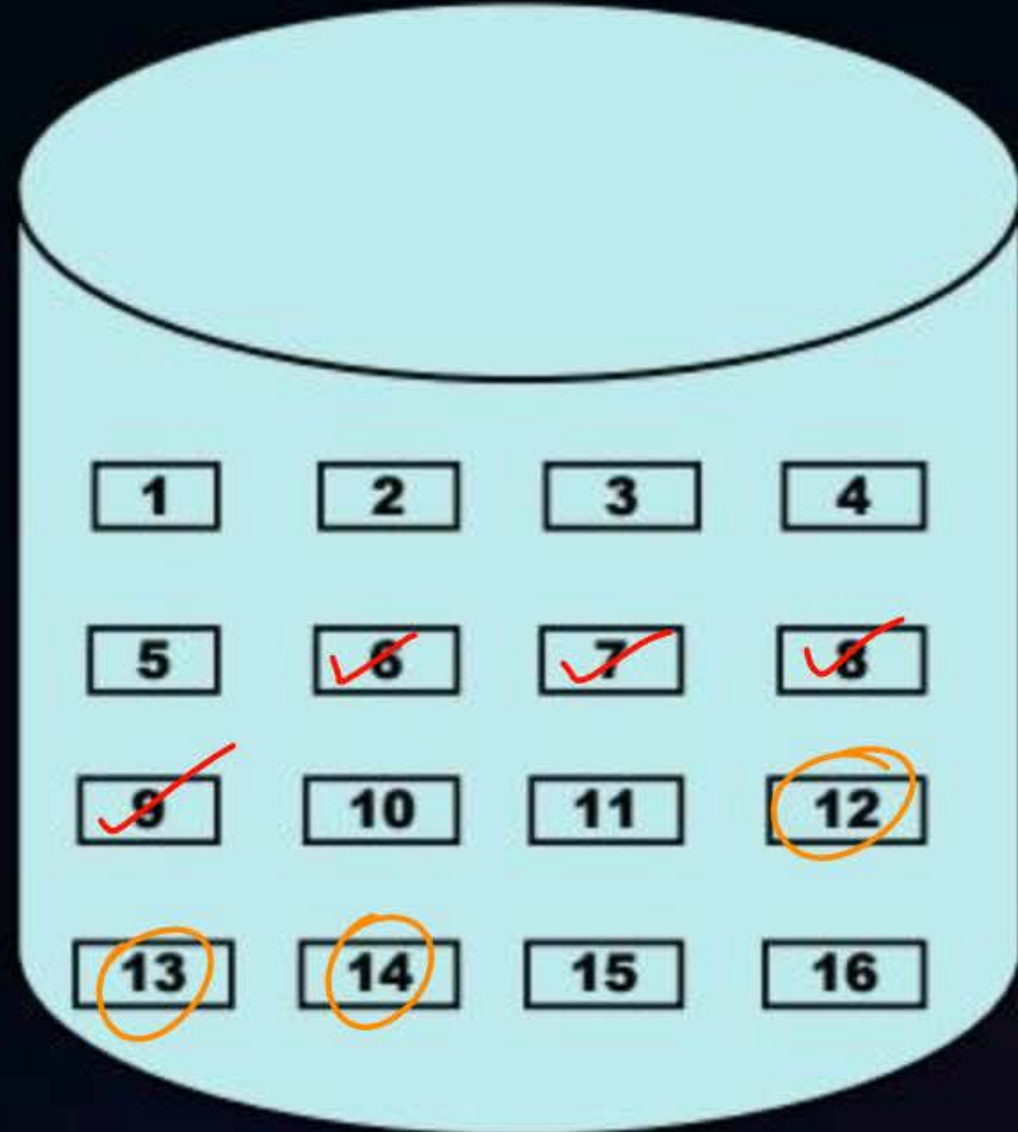
Topic : File Allocation Method

1. Contiguous Allocation
2. Linked Allocation
3. Indexed Allocation



Topic : Contiguous Allocation

↓
consecutive blocks are allocated to file.



File allocation Table

file name	starting block no.	Total no. of blocks in file
os.pdf	6	4
COA.pptx	12	3



Topic : Contiguous Allocation

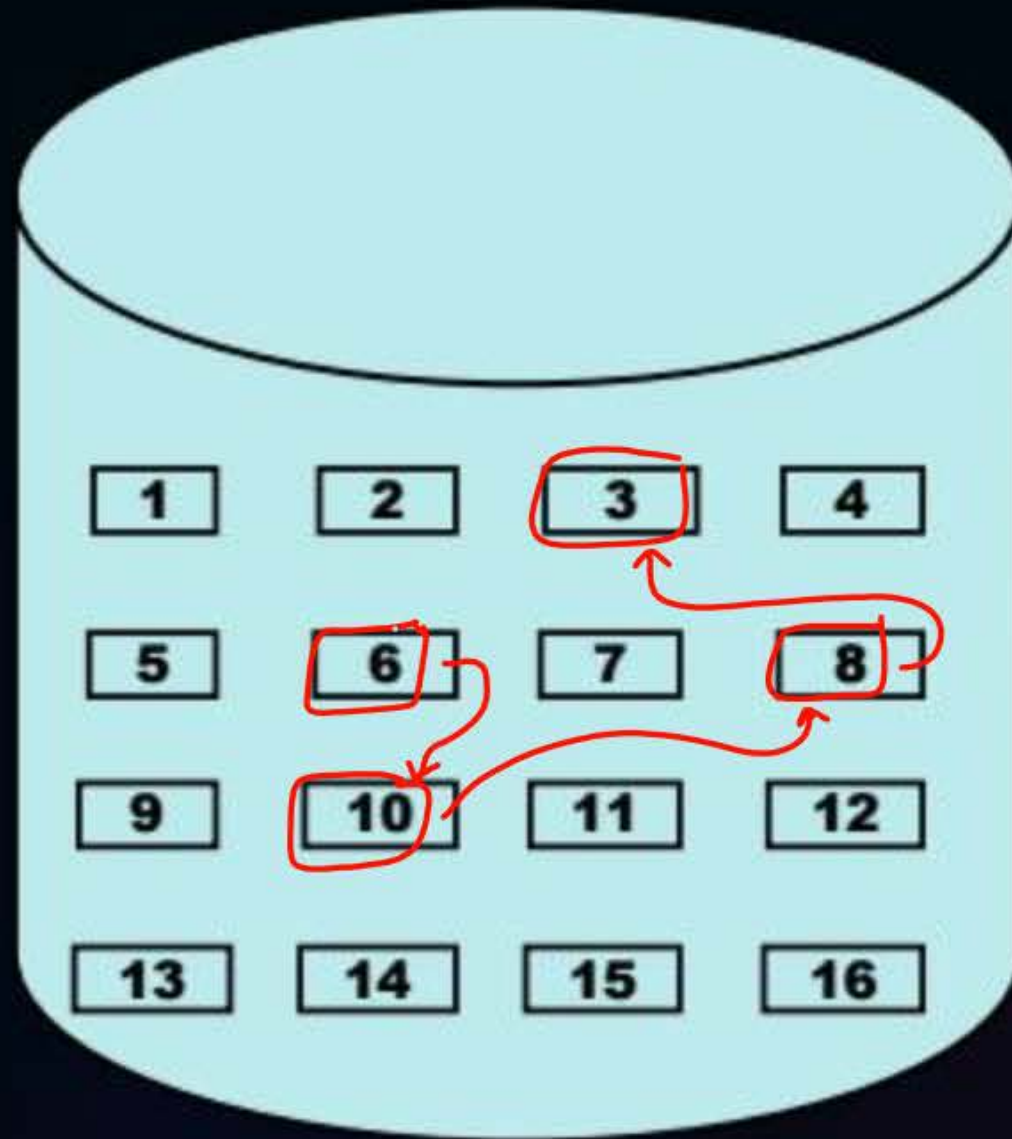
Performance:

1. Fragmentation: Internal, External → disadvantages
2. Increase in File size: Inflexible → if consecutive blocks around file are not available
3. Type of access: Sequential, Random/direct → advantage

Insertion of a new block in file at	Required no. of block accesses
Beginning	Too many
Middle	Too many
End	Limited



Topic : Linked Allocation



	start block no.	last block no.
os.pdf	6	3



Topic : Linked Allocation

Performance:

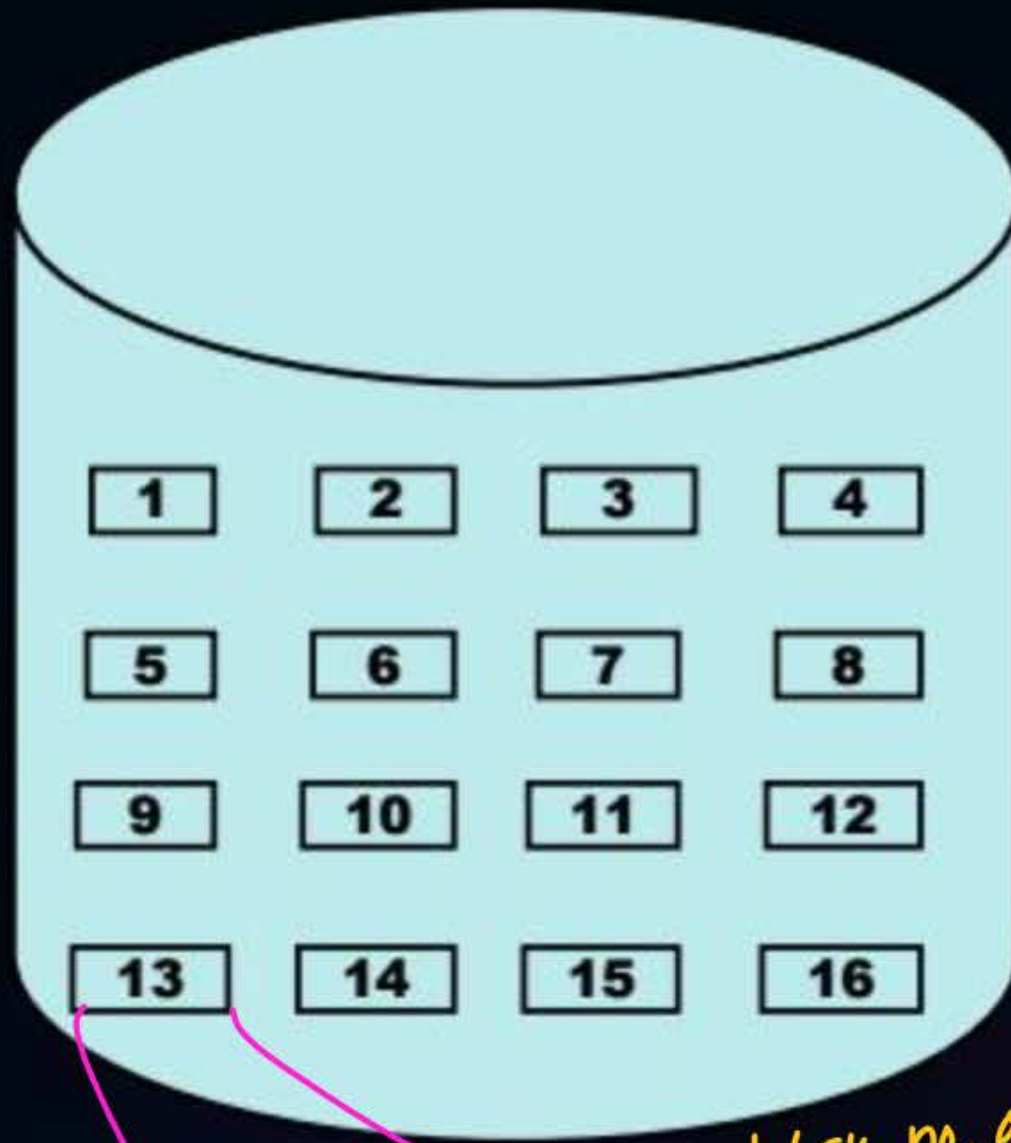
1. Fragmentation: Internal
2. Increase in File size: Flexible
3. Type of access: Sequential

Disadv:- No direct access of file block
adv:- No external fragmentation

Insertion of a new block in file at	no. of block access required
Beginning	Limited
Middle	Too many
End	Too many



Topic : Indexed Allocation



File name	Index block no.
OS.pdf	13

6
10
8
3

block no. of blocks
where file is stored



Topic : Indexed Allocation

Performance:

1. Fragmentation: Internal
2. Increase in File size: Flexible
3. Type of access: Sequential, Random/direct

all adv.

Insertion of a new block in file at	no. of block access required
Beginning	} limited
Middle	
End	

Disadv:-
one extra block occupied to store index.



Topic : Question

H.W.

[GATE-2022]



#Q. Consider two files systems A and B , that use contiguous allocation and linked allocation, respectively. A file of size 100 blocks is already stored in A and also in B. Now, consider inserting a new block in the middle of the file (between 50th and 51st block), whose data is already available in the memory. Assume that there are enough free blocks at the end of the file and that the file control blocks are already in memory. Let the number of disk accesses required to insert a block in the middle of the file in A and B are n_A and n_B respectively, then the value of $n_A + n_B$ is_____?



Topic : Question



- Disk block address = 16 bits 2 Bytes
- Disk block size = 1KB
- Index block = 1KB
- Maximum ~~size~~? no. of indexes kept in single disk block?

$$= \frac{\text{block size}}{\text{D.B. address}} = \frac{1KB}{2B} = 2^9$$

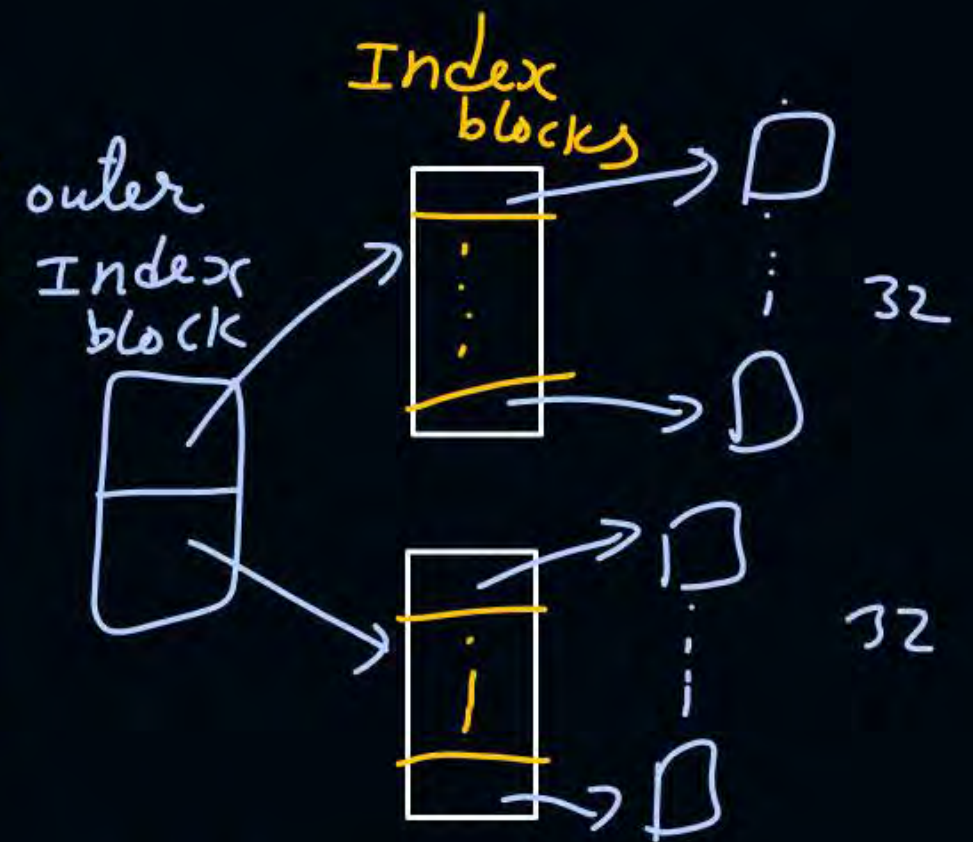
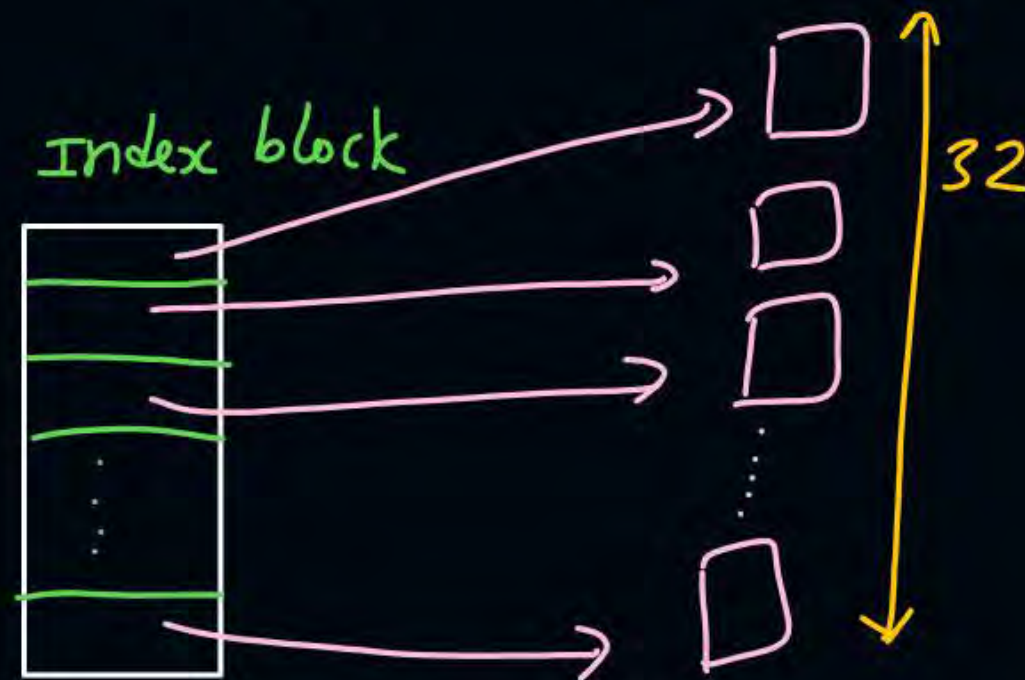
ex:-

DB Address = 32 bits = 4B

Index block = 128B

$$\text{no. of indexes per block} = \frac{128B}{4B} = 32$$

assume:- A file is stored on 64 blocks

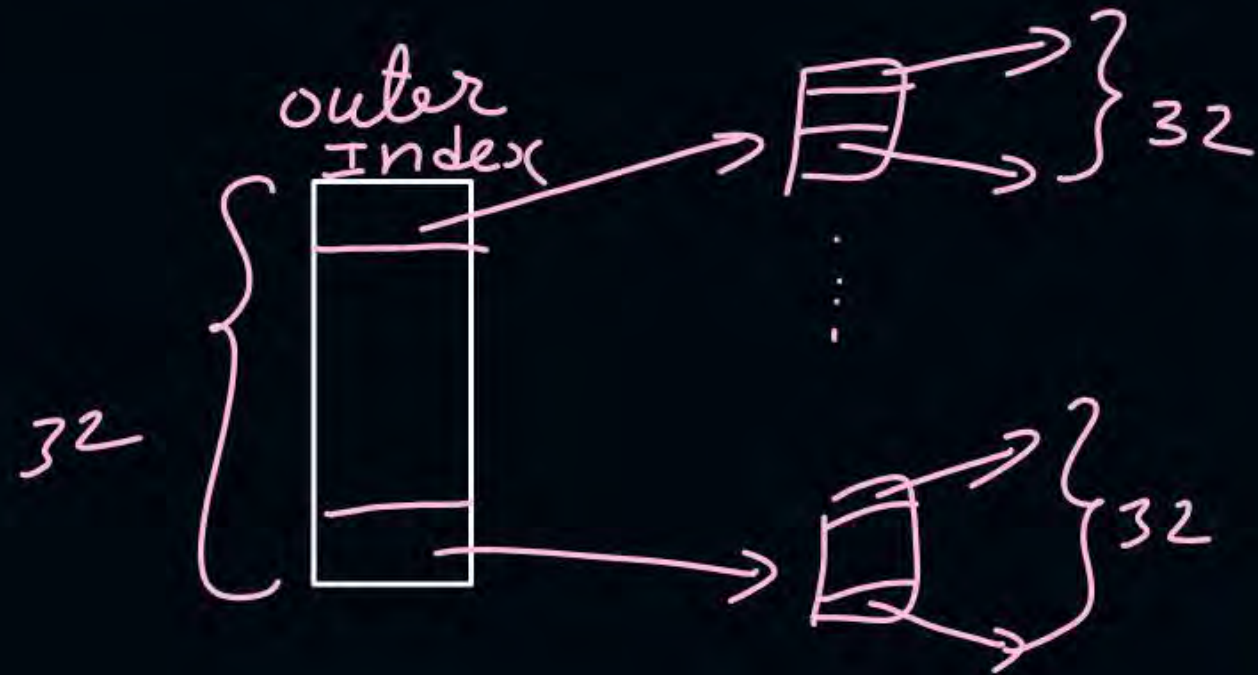


2-level indexing

for prev. example:-

2-level indexing

max no. of blocks to store file = $32 * 32 = 1024$



3 level indexing:-

max no. of blocks to store file

$$= 2^5 * 2^5 * 2^5$$
$$= 2^{15}$$



Topic : Master Boot Record

A master boot record (MBR) is a special type of boot sector at the very beginning of partitioned computer mass storage devices.

- Contains the information regarding how and where the OS is located in the hard disk so that it can be booted in the RAM.



Topic : Unix I-node Structure

→ index node

The inode (index node) is a data structure in a Unix-style file system that describes a file-system object such as a file or a directory.

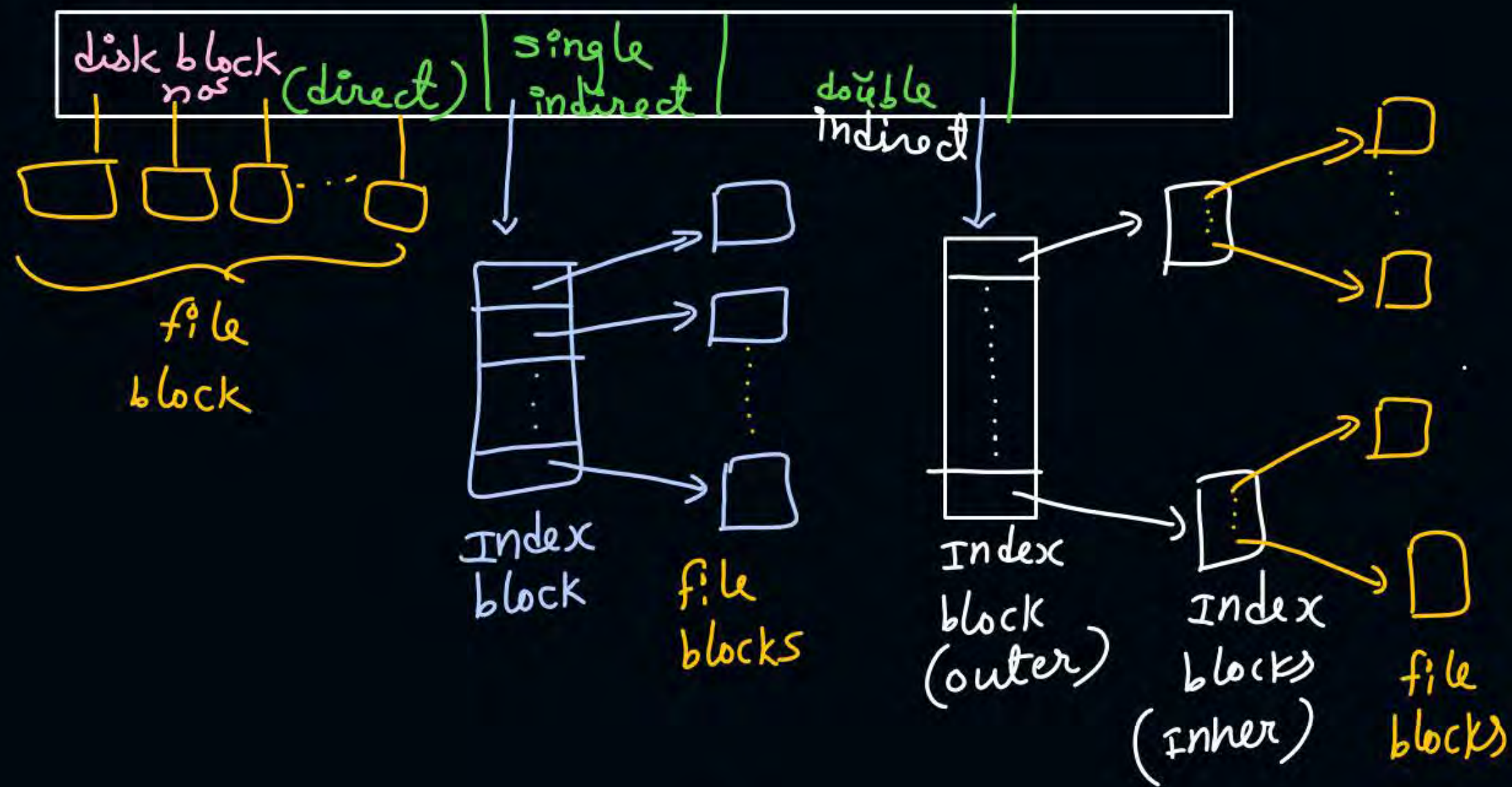


Topic : Unix I-node Structure

The inode (index node) is a data structure in a Unix-style file system that describes a file-system object such as a file or a directory.

- Each inode stores the attributes and disk block locations of the object's data.
- The number of Inode limits the total number of files/directories that can be stored in the file system.

i-node





Topic : Question

[GATE-2019]



#Q. The index node (inode) of a Unix-like file system has 12 direct, one single-indirect and one double-indirect pointer. The disk block size is 4 kB, and the disk block addresses 32-bits long. The maximum possible file size is (rounded off to 1 decimal place) 4.0 GB?

→ 4B

$$\begin{aligned}\text{no. of indexes per block} &= \frac{4\text{KB}}{4\text{B}} \\ &= 1\text{k} \\ &= 2^{10}\end{aligned}$$

$$\begin{aligned}\text{Max file size} &= (12 * 4\text{KB}) + (2^{10} * 4\text{KB}) \\ &\quad + 2^{10} * 2^{10} * 4\text{KB} \\ &\text{(use all pointers)}\end{aligned}$$

$$\begin{aligned}&= 48\text{KB} + 4\text{MB} + 4\text{GB} \\ &= 0.000048\text{GB} + 0.004\text{GB} + 4\text{GB} \\ &= 4.004048\text{GB}\end{aligned}$$



Topic : Question

[GATE-2005]



#Q. In a computer system, four files of size 11050 bytes, 4990 bytes, 5170 bytes and 12640 bytes need to be stored. For storing these files on disk, we can use either 100 byte disk blocks or 200 byte disk blocks (but can't mix block sizes). For each block used to store a file, 4 bytes of bookkeeping information also needs to be stored on the disk. Thus, the total space used to store a file is the sum of the space taken to store the file and the space taken to store the book keeping information for the blocks allocated for storing the file. A disk block can store either bookkeeping information for a file or data from a file, but not both.

What is the total space required for storing the files using 100 byte disk blocks and 200 byte disk blocks respectively?



35400 and 35800 bytes



35800 and 35400 bytes



35600 and 35400 bytes



35400 and 35600 bytes



Topic : Solution

Blocks of 100 Bytes



File Size	No. of blocks to store file	No. of blocks to store bookkeeping	Total blocks	Total Size
11050 Bytes	$\left\lceil \frac{11050 \text{ B}}{100 \text{ B}} \right\rceil = 111$	$\left\lceil \frac{111 * 4 \text{ B}}{100 \text{ B}} \right\rceil = 5$	116	
4990 Bytes	$= 50$	$\left\lceil \frac{50 * 4 \text{ B}}{100 \text{ B}} \right\rceil = 2$	52	
5170 Bytes	$= 52$	$\left\lceil \frac{52 * 4 \text{ B}}{100 \text{ B}} \right\rceil = 3$	55	
12640 Bytes	$= 127$	$\left\lceil \frac{127 * 4 \text{ B}}{100 \text{ B}} \right\rceil = 6$	133	

Total blocks 356

$$\begin{aligned}\text{Total space} &= 356 * 100 \text{ B} \\ &= 35600 \text{ B}\end{aligned}$$



Topic : Solution

Blocks $\Rightarrow 200\text{B}$

File Size	No. of blocks to store file	No. of blocks to store bookkeeping	Total blocks	Total Size
11050 Bytes	56	$\left\lceil \frac{56 * 4\text{B}}{200\text{B}} \right\rceil = 2$	58	
4990 Bytes	25	$\left\lceil \frac{25 * 4\text{B}}{200\text{B}} \right\rceil = 1$	26	
5170 Bytes	26	$\left\lceil \frac{26 * 4\text{B}}{200\text{B}} \right\rceil = 1$	27	
12640 Bytes	64	$\left\lceil \frac{64 * 4\text{B}}{200\text{B}} \right\rceil = 2$	66	

Total blocks = 177

$$\text{Total size} = 177 * 200\text{B} = 35400\text{B}$$



2 mins Summary

Topic

Disk Blocks

Topic

File Allocation Method

Topic

Disk Scheduling



Happy Learning

THANK - YOU